Kế Hoạch Phát Triển Bài Báo Khoa Học (System / Tool Paper Plan)

1. Định Vị & Đóng Góp Cốt Lõi (Positioning & Core Contributions)

Dự án của bạn không chỉ đơn thuần là một Moodle plugin mở rộng giao diện, mà là một **Hệ thống Học tập Kiểm thử Vi sai & Trao đổi Testcase Cộng đồng (Differential Testing & Peer Testcase Exchange System)** tích hợp trực tiếp vào hệ sinh thái LMS (Moodle + CodeRunner + Jobe Cluster).

Bài toán thực tiễn (Problem Statement)

1. **Hội chứng phụ thuộc bộ chấm tự động (Autograder Over-reliance / Trial-and-Error)**: Trong các khóa học lập trình sử dụng CodeRunner, sinh viên thường nộp bài mù quáng để thử vận may với các hidden testcases thay vì tự rèn luyện tư duy thiết kế testcase (Test-Driven Thinking).
2. **Rò rỉ đáp án chuẩn (Oracle Leakage)**: Các công cụ cho phép sinh viên thử test input trên bài giải mẫu của giảng viên (Reference Solution / Oracle) thường làm lộ trực tiếp kết quả đầu ra (`stdout`), tạo kẽ hở để sinh viên "dò đáp án" (Oracle Gaming) hoặc hard-code kết quả.
3. **Chi phí tạo bộ kiểm thử biên của Giảng viên & Quá tải Sandbox**: Giảng viên tốn nhiều thời gian nghĩ các trường hợp biên (boundary/edge cases), trong khi việc chạy đồng thời code sinh viên và code mẫu trên Jobe dễ gây nghẽn cổ chai (bottleneck) và bị spam.

4 Đóng góp Khoa học & Kỹ thuật (Key Contributions)

1. **Kiến trúc Kiểm thử Vi sai Bảo mật Đáp án (Zero-Leakage Differential Testing Playground)**:
    - Cho phép sinh viên tự đặt giả thuyết (Expected Output) và đối chiếu giữa *Code của mình* với *Reference Solution* của giảng viên.
    - Cơ chế **Oracle Masking & Prediction Gating**: Ẩn hoàn toàn đầu ra thực tế của Solution; chỉ xác nhận Đúng/Sai chuẩn đề và chỉ mở khóa tính năng đóng góp khi dự đoán của sinh viên khớp chính xác với Solution.
2. **Cơ chế Đóng góp & Đổi thưởng Testcase Đồng đẳng (Reciprocal Peer Testcase Crowdsourcing & Gamification)**:

    - Mô hình kinh tế chia sẻ (Give-to-Get): Sinh viên đóng góp testcase chất lượng (được duyệt tự động hoặc qua thẩm định 1–5 sao của giảng viên) sẽ nhận được lượt mở khóa các testcase hiểm hóc từ kho chung (Pool) hoặc từ Hạt giống của giảng viên (Seed Bank).

3. **Tối ưu Hóa Hạ tầng Thực thi & Chống Trùng Lặp Đa tầng (Multi-tier Deduplication & Sandbox Optimization)**:
    - Chuẩn hóa đầu vào (Normalization) và định danh bằng **SHA-256 Canonical Fingerprinting** (`testcase_canonical_keys`).
    - **100% Sandbox Bypass** cho các lượt chạy trùng lặp `(user, question, fingerprint, code_hash)` và **50% Compute Reduction** nhờ cơ chế **Oracle Output Caching** trên cụm Jobe song song (`jobe1`, `jobe2`).
4. **Hệ thống Truy vết Hành vi Học tập theo Trục Thời gian (Fine-grained Learning Analytics & Code Evolution Tracking)**:
    - Ghi nhận tiến trình phát triển mã nguồn theo chuỗi chuyển trạng thái liền kề ($v_1 \to v_2 \to \cdots \to v_k$ v1 → v2 → ⋯ → vk) gắn kết chặt chẽ với6 sự kiện học tập trên trục thời gian hợp nhất (Unified Chronological Timeline), sẵn sàng xuất dữ liệu chuẩn cho nghiên cứu giáo dục (EDM / Learning Analytics).

2. Gợi Ý Tên Bài Báo (Candidate Titles)

- **Hướng Công nghệ Giáo dục (CSE / EdTech - SIGCSE, ITiCSE, IEEE TLT):**

  > *"TestcaseExchange: A Zero-Leakage Differential Testing and Peer Testcase Crowdsourcing Plugin for Moodle CodeRunner"*

- **Hướng Công nghệ Phần mềm & Công cụ (SE Tool Track - ICSE SEET, FSE, ASE):**

  > *"From Trial-and-Error to Test-Driven Learning: A Moodle-Jobe System for Reciprocal Testcase Exchange and Code Evolution Analytics"*

3. Cấu Trúc Bài Báo Chi Tiết (6–8 trang chuẩn IEEE / ACM)

Chi tiết từng mục:

Section 1: Introduction (1 trang)

- Đặt vấn đề: Sự phổ biến của Moodle và CodeRunner trong giảng dạy lập trình (CS1/CS2).
- Khoảng trống nghiên cứu (Research Gap): Thiếu công cụ tích hợp sẵn trong Moodle cho phép kiểm thử vi sai (Differential Testing) mà không làm lộ output của thầy cô, đồng thời khuyến khích sinh viên chia sẻ testcase.
- Tóm tắt 4 đóng góp chính của bài báo.

Section 2: Related Work (0.75 - 1 trang)

- So sánh hệ thống của bạn với các công cụ hiện có bằng một **Bảng đối sánh tính năng (Feature Comparison Matrix)**:
  - *CodeRunner chuẩn*: Chỉ chấm điểm, không có Playground vi sai hay trao đổi testcase.
  - *PrairieLearn / Web-CAT*: Yêu cầu sinh viên nộp kèm test nhưng thiết lập phức tạp, tách rời Moodle hoặc dễ lộ hành vi của reference solution.
  - *Code Defenders / Peer Testing tools*: Thường là hệ thống rời (standalone), không đồng bộ trực tiếp với Question Bank, Quiz Access Rule và Gradebook của Moodle.

Section 3: System Architecture & Pedagogical Workflow (1.5 trang)

- **Sơ đồ kiến trúc tổng thể**:
  - Tầng LMS: Moodle Core + `qtype_coderunner` + Subplugin `quizaccess_testcaseexchange` + Local plugin `local_testcase_exchange`.
  - Tầng Lưu trữ Độc lập: Cơ sở dữ liệu `testcase_store` (MariaDB) thiết kế theo phân cấp 3 tầng (`course_id` → → `quiz_id` → → `question_id`) tách biệt khỏi DB lõi của Moodle để đảm bảo hiệu năng và tính toàn vẹn khi nâng cấp.
  - Tầng Thực thi Cách ly: Cụm Jobe Sandbox song song (`jobe1:4001`, `jobe2:4002`).
- **Chu trình sư phạm 4 giai đoạn (4-Stage Pedagogical Loop)**:
  1. *Explore & Hypothesize*: Viết code, tự thiết kế input và tự dự đoán kết quả (Expected Output).
  2. *Differential Verify*: Chạy đối chiếu ẩn danh với Solution của giảng viên.
  3. *Contribute & Review*: Đề xuất testcase đúng chuẩn vào kho chung; giảng viên duyệt/chấm sao hoặc auto-approve.
  4. *Unlock & Improve*: Nhận thưởng testcase từ kho chung để phát hiện lỗi ẩn trong code trước khi nộp bài Quiz chính thức.

Section 4: Core Technical & Security Mechanisms (1.5 trang)

- **Zero-Leakage Oracle Verification**: Trình bày công thức/logic kiểm tra tam giác $(O_{\text{student}}, O_{\text{expected}}, O_{\text{oracle}})$(Ostudent, Oexpected, Ooracle). Giải thích vì sao ẩn O_{oracle} Ooracle và bắt buộc $O_{\text{expected}} = = O_{\text{oracle}}$ Oexpected == Ooracle mới cho phép đóng góp giúp triệt tiêu hoàn toàn hành vi dò đáp án.
- **Canonical Fingerprinting & Proactive Deduplication**: Chuẩn hóa đa chế độ (`trim` , `json` key-sorting) → → Băm `SHA-256` → → Ràng buộc nguyên tử trên `testcase_canonical_keys` .
- **Sandbox Load Optimization**: Cơ chế chặn chạy trùng 100% và bộ nhớ đệm `Oracle Output Caching` giảm 50% số lệnh gọi sang Jobe.

Section 5: Code Evolution & Chronological Learning Analytics (0.75 trang)

- Mô hình lưu vết phiên bản mã nguồn theo sự thay đổi liền kề $(H(C_t) \neq H(C_{t-1})$H (Ct) ≣= H (Ct–1)) kết hợp lưu trữ tạm thời (LocalStorage Persistence) chống mất trạng thái làm bài.
- Giới thiệu công cụ trích xuất trục thời gian hợp nhất (6 loại sự kiện: `PLAYGROUND_TEST` , `TESTCASE_CONTRIBUTE` , `TESTCASE_REVIEW` , `TESTCASE_REWARD` , `QUIZ_SUBMIT`), hỗ trợ xuất CSV/JSON trực tiếp phục vụ nghiên cứu Khai phá dữ liệu giáo dục (Educational Data Mining).

Section 6: Evaluation (1.5 trang) — *Phần quan trọng nhất để bài báo được nhận!*

Một bài System/Tool Paper mạnh cần **2 phần đánh giá**:

1. **Đánh giá Hiệu năng Kỹ thuật (System Performance & Scalability Benchmark)**:
    - Đo lường tỷ lệ tiết kiệm tài nguyên trên cụm Jobe (`jobe1` , `jobe2`) khi bật **Anti-Spam Deduplication** và **Oracle Output Caching**.
    - *Kịch bản mô phỏng*: Cho 50–100 sinh viên ảo (hoặc dữ liệu pilot) gửi 1.000 lượt chạy thử nghiệm. So sánh số lượng Jobe API calls, thời gian phản hồi trung bình (Latency ms) và tải CPU khi *Không có Cache/Deduplication* vs. *Có Cache/Deduplication* (thường giảm từ 45%–70% số lệnh thực thi).
2. **Đánh giá Sư phạm / Thực nghiệm (Pilot Study / Case Study)**:
    - Triển khai trên 1–2 lớp học thực tế (ví dụ: môn Lập trình hướng đối tượng / Cấu trúc dữ liệu với khóa học hiện tại của bạn).
    - **Các chỉ số định lượng trích xuất từ chính script `export_student_timeline.py` **:
        - Số phiên bản code trung bình mỗi sinh viên trải qua trước khi nộp bài Quiz chính thức (ví dụ: trung bình từ `Code v1` đến `Code v5`).
        - Tỷ lệ testcase đa dạng được sinh viên đóng góp (Normal vs. Boundary vs. Edge/Exception cases).

        - Tương quan giữa số lần thử nghiệm trong Contribute Zone và điểm số bài thi Quiz chính thức (`QUIZ_SUBMIT`).
    - **Khảo sát ngắn (System Usability Scale - SUS / Likert 5 mức)**: Khảo sát sinh viên về tính dễ sử dụng (nhờ lưu trạng thái tự động, nạp lại code cũ, giao diện rõ ràng).

Section 7: Conclusion & Open-Source Availability (0.5 trang)

- Tổng kết đóng góp, hướng phát triển tiếp theo (ví dụ: tích hợp LLM gợi ý nhãn phân loại testcase hoặc gợi ý phản hồi cho giảng viên).
- Cung cấp link GitHub repository (hoặc Zenodo DOI) kèm Docker Compose để hội đồng phản biện có thể dựng lại toàn bộ môi trường demo chỉ bằng 1 câu lệnh (Artifact Evaluation Badge).

4. Các Hội Nghị & Tạp Chí Mục Tiêu Phù Hợp Nhất

Phân nhóm	Tên Hội nghị / Tạp chí	Xếp hạng / Uy tín	Đặc điểm phù hợp
Hội nghị Giáo dục CNTT (Top-tier)	**ACM SIGCSE TS** / **ACM ITiCSE** / **IEEE FIE**	CORE A / Top EdTech	Rất chuộng các công cụ tích hợp vào Moodle/CodeRunner có giải pháp sư phạm rõ ràng và đánh giá trên lớp học.
Hội nghị Công nghệ Phần mềm (Tool / SEET Track)	**ICSE (SEET / Demo Track)** / **ESEC/FSE** / **ASE**	CORE A* (Tool/Education Track)	Đánh giá cao kỹ thuật Differential Testing, Fingerprinting, Sandbox Caching và khả năng tái lập qua Docker.
Tạp chí Quốc tế (SCI/Scopus Q1/Q2)	**SoftwareX** (Elsevier) / **Journal of Open Source Software (JOSS)**	Scopus Q1/Q2 (Chuyên về Tool/Software)	Bài báo ngắn gọn (4–6 trang), tập trung vào kiến trúc phần mềm, mã nguồn sạch và tính ứng dụng thực tế.
Tạp chí Giáo dục Kỹ thuật (Q1)	**IEEE Transactions on Learning Technologies (TLT)** / **Computer Applications in Engineering Education (Wiley)**	ISI/Scopus Q1	Phù hợp nếu bạn có dữ liệu thực nghiệm khoảng 1 học kỳ (từ 40–100 sinh viên) phân tích sâu qua Timeline Exporter.

5. Lộ Trình Thực Hiện Đề Xuất (Actionable Roadmap)

1. **Giai đoạn 1 (1 tuần) — Đóng gói Artifact & Benchmark Kỹ thuật**:
    - Viết script benchmark tự động đo hiệu năng Jobe Cluster (so sánh trước và sau khi bật Oracle Caching & Duplicate Run Blocking) để lấy bảng số liệu và biểu đồ cho Section 6.1.
2. **Giai đoạn 2 (2–3 tuần) — Thu thập dữ liệu lớp học (Pilot Data)**:
    - Cho sinh viên sử dụng trên các bài Quiz thực tế.
    - Dùng `scripts/export_student_timeline.py` xuất toàn bộ chuỗi sự kiện ra CSV/JSON và chạy script thống kê hành vi tiến hóa code (`Code v1 -> vn`).
3. **Giai đoạn 3 (2 tuần) — Viết bản thảo LaTeX (ACM / IEEE Template)**:
    - Chuyển hóa các sơ đồ kiến trúc, bảng ánh xạ DB và kết quả thực nghiệm thành bản thảo hoàn chỉnh.